1D Array Basics & Operations

Question: Define an array and discuss any two types/ways of initialization.

Answer: An array is a fixed-size sequenced collection of homogeneous elements of the same data type that are referenced by a common name. Array elements are stored sequentially one after the other in memory.

The document outlines several ways to initialize a single-dimensional array, including:

- **Static Initialization:** This method initializes the array at the time of declaration using a list of values enclosed in curly braces.

```c
C
int b[4] = {10, 12, 14, 16};
```

- Note: If you provide more elements than the declared size, the compiler will throw a "too many initializers" error.
- **Partial Initialization:** If the number of values provided in the initializer list is less than the declared size of the array, it is called partial initialization. The remaining positions are automatically initialized to zero or a null character ('\0').

```c
C
int b[4] = {10, 12}; // b[2] and b[3] automatically become 0
```

Question: Develop a C program to add two one-dimensional arrays.

Answer: (Note: The source document on Page 28 contains broken text and lacks standard formatting. Below is the cleaned up, corrected version that cleanly reads two arrays and stores their element-wise sum into a third array.)

```c
#include <stdio.h>

int main() {
    int a[20], b[20], c[20], n, i;

    printf("Enter the number of elements\n");
    scanf("%d", &n);

    printf("Enter the elements of Array A\n");
    for (i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }
```

```
    printf("Enter the elements of Array B\n");
    for (i = 0; i < n; i++) {
        scanf("%d", &b[i]);
    }

    // Array Addition
    for (i = 0; i < n; i++) {
        c[i] = a[i] + b[i];
    }

    printf("The resultant array is \n");
    for (i = 0; i < n; i++) {
        printf("%d\n", c[i]);
    }

    return 0;
}
```

Question: Develop a C program to find the sum of 1-D array elements.

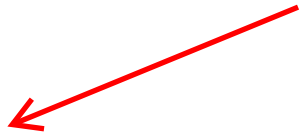
Answer:

(Note: The program on Page 34 of your PDF titled "find sum of n Natural numbers" contains a critical logical bug. It declares an array a[10] but completely ignores it, instead performing a loop that sums the index values sum = sum + i. Below is the corrected program that actually populates a 1-D array with user input and finds the sum of its elements.)

C

```c
#include <stdio.h>

int main() {
    int a[100], n, i, sum = 0;

    printf("Enter the number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    // Correct logic: looping through and summing the actual array
```

```
elements
    for (i = 0; i < n; i++) {
        sum = sum + a[i];
    }

    printf("Sum of array elements = %d\n", sum);

    return 0;
}
```



Question: Construct a C program to read 'n' elements in an array and print the same in reverse order.

Answer:

(Note: The program on Page 31 has fragmented text and a broken loop syntax or(i=n-1;i>=0;i...). Below is the corrected version.)

C

```
#include <stdio.h>

int main() {
    int a[20], n, i;

    printf("Enter the array size: ");
    scanf("%d", &n);

    printf("Enter the array elements\n");
    for (i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    printf("The elements entered in the array are:\n");
    for (i = 0; i < n; i++) {
        printf("%d\t", a[i]);
    }
    printf("\n");

    printf("The elements of the array in reverse order are:\n");
    for (i = n - 1; i >= 0; i--) {
        printf("%d\t", a[i]);
    }
    printf("\n");

    return 0;
}
```

Linear Search

Question: Develop a C program which reads 'n' elements (sorted or unsorted) in an array and searches for a key element using the linear search technique.

Answer:

(Note: The program on Page 43 has minor textual layout fragmentation but is logically

```
functional. The corrected code structure is cleaned up below.)
C
#include <stdio.h>

int main() {
    int a[100], n, i, key, flag = 0;

    printf("Enter the no of elements\n");
    scanf("%d", &n);

    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    printf("Enter the element to be searched\n");
    scanf("%d", &key);

    // Linear search execution
    for (i = 0; i < n; i++) {
        if (key == a[i]) {
            flag = 1;
            break;
        }
    }

    if (flag == 1) {
        printf("Element found at position %d\n", i + 1);
    } else {
        printf("Element not found\n");
    }

    return 0;
}
```

Question: Define Searching.

Answer: Searching is the process of finding the position of a particular target element (often called the key element) within a large collection of data. If the key element matches an item in the list, the search is considered successful; otherwise, it is unsuccessful.

2D Arrays & Matrix Operations

Question: Define 2-D array.

Answer: A two-dimensional (2-D) array is the simplest form of a multidimensional array, which stores data in the form of a grid consisting of rows and columns. It is characterized by having two subscripts:

- The **first subscript** represents the number of rows.
- The **second subscript** represents the number of columns.

All elements in a 2-D array share a common name and must be of the same homogeneous data type.

Syntax:

```c
data_type array_name[row_size][column_size];
```

Question: Develop a C program to find the addition of two matrices.

Answer: (Note: The source document on Page 56 has scattered loop declarations and mismatched braces. The corrected and cleanly structured version is provided below.)

```c
#include <stdio.h>

int main() {
    int a[20][20], b[20][20], c[20][20], m, n, i, j;

    printf("enter the rows and column of matrix\n");
    scanf("%d%d", &m, &n);

    printf("Enter the elements of Matrix A\n");
    for (i = 0; i < m; i++) {
        for (j = 0; j < n; j++) {
            scanf("%d", &a[i][j]);
        }
    }

    printf("Enter the elements of Matrix B\n");
    for (i = 0; i < m; i++) {
```

```
        for (j = 0; j < n; j++) {
            scanf("%d", &b[i][j]);
        }
    }

    // Matrix Addition Logic
    for (i = 0; i < m; i++) {
        for (j = 0; j < n; j++) {
            c[i][j] = a[i][j] + b[i][j];
        }
    }

    printf("Matrix Addition\n");
    printf("The resultant matrix is\n");
    for (i = 0; i < m; i++) {
        for (j = 0; j < n; j++) {
            printf("%d\t", c[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

Question: Develop a C program to find the transpose of a given matrix.

Answer: (Note: The code on Page 58 of the PDF contains multiple syntax bugs, including a typo in the variable declarations m.n, i j , missing initialization loops, and mixed up row/column index boundaries. Below is the completely functional corrected code.)

```
C
#include <stdio.h>

int main() {
    int a[20][20], b[20][20], m, n, i, j;

    printf("Matrix Transpose\n");
    printf("enter the rows and column of matrix\n");
    scanf("%d%d", &m, &n);

    printf("Enter the elements of Matrix\n");
    for (i = 0; i < m; i++) {
        for (j = 0; j < n; j++) {
```

```
            scanf("%d", &a[i][j]);
        }
    }

    // Transpose Logic: Rows become columns
    for (i = 0; i < m; i++) {
        for (j = 0; j < n; j++) {
            b[j][i] = a[i][j];
        }
    }

    printf("The Transpose of the matrix is \n");
    // Loop boundaries switch to 'n' rows and 'm' columns for the
transposed matrix
    for (i = 0; i < n; i++) {
        for (j = 0; j < m; j++) {
            printf("%d\t", b[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

String Manipulation (Theory & Manual Implementations)

Question: Demonstrate any 3 string manipulation functions in C with syntax.

Answer: The following are three standard string handling functions available in the <string.h> header file:

1. **strlen()**
    - **Purpose:** Returns the total number of characters in a given string, excluding the terminal null character.
    - **Syntax:** strlen(string_data);
2. **strcpy()**
    - **Purpose:** Copies the complete content of a source string into a destination string variable.
    - **Syntax:** strcpy(destination_string, source_string);
3. **strcat()**
    - **Purpose:** Concatenates (joins) the source string onto the tail end of the destination string.
    - **Syntax:** strcat(destination_string, source_string);

Question: Develop a C program to compare two strings using a string manipulation function.

Answer: (Note: Page 78 explicitly features a program using strcmp(). It has been minorly restructured for clean execution.)

```
C
#include <stdio.h>
#include <string.h>

int main() {
    char str1[20], str2[20];
    int k;

    printf("Enter string 1\n");
    scanf("%s", str1);

    printf("Enter string 2\n");
    scanf("%s", str2);

    k = strcmp(str1, str2);

    if (k == 0) {
        printf("Strings are same\n");
    } else {
        printf("Strings are different\n");
    }
    return 0;
}
```

Question: Develop a C program to concatenate/join two strings without using built-in string manipulation functions.

Answer: (Note: The manual implementation provided on Page 85 contains logic errors. It misses tracking the length destination marker step correctly (k remains stuck or unaligned). Here is the industry-standard way to complete this manual concatenation dynamically.)

C

```
#include <stdio.h>

int main() {
    char str1[40], str2[20]; // str1 size needs to be large enough to
hold both strings
    int i = 0, j = 0;

    printf("Enter String1\n");
    scanf("%s", str1);

    printf("Enter String2\n");
```

```
    scanf("%s", str2);

    // Step 1: Move index 'i' to the end of str1 (the null character
position)
    while (str1[i] != '\0') {
        i++;
    }

    // Step 2: Copy characters of str2 into str1 starting at index 'i'
    while (str2[j] != '\0') {
        str1[i] = str2[j];
        i++;
        j++;
    }

    // Step 3: Explicitly null-terminate the concatenated string
    str1[i] = '\0';

    printf("The concatenated string is=%s\n", str1);
    return 0;
}
```

Question: Develop a C program to compare two strings without using built-in string manipulation functions.

Answer: (Note: The manual implementation shown on Pages 79 and 80 uses strlen() to get sizes first, which violates a strict "without any built-in functions" requirement. Below is the fully manual iteration technique that tests characters lineally without using strlen() or strcmp().)

C

```
#include <stdio.h>

int main() {
    char str1[20], str2[20];
    int i = 0, flag = 0;

    printf("Enter string 1\n");
    scanf("%s", str1);

    printf("Enter string 2\n");
    scanf("%s", str2);

    // Compare strings character by character until a mismatch or
terminal null is hit
    while (str1[i] != '\0' || str2[i] != '\0') {
```

```
        if (str1[i] != str2[i]) {
            flag = 1;
            break;
        }
        i++;
    }

    if (flag == 0 && str1[i] == '\0' && str2[i] == '\0') {
        printf("Strings are same\n");
    } else {
        printf("Strings are different\n");
    }

    return 0;
}
```

Medium Priority: String Sorting & Palindrome

Question: Develop a C program to read N names and sort the names in alphabetic order.

Answer: (Note: While the document explains 2D arrays of strings on Pages 71–73, it does not provide a full name-sorting program. Below is a correct C program that reads N names and uses the Bubble Sort algorithm to arrange them alphabetically.)

```
C
#include <stdio.h>
#include <string.h>

int main() {
    char names[50][50], temp[50];
    int n, i, j;

    printf("Enter the number of names: ");
    scanf("%d", &n);

    printf("Enter %d names:\n", n);
    for (i = 0; i < n; i++) {
        scanf("%s", names[i]);
    }

    // Bubble sort logic for sorting strings alphabetically
    for (i = 0; i < n - 1; i++) {
```

```
        for (j = 0; j < n - i - 1; j++) {
            if (strcmp(names[j], names[j + 1]) > 0) {
                // Swap the strings
                strcpy(temp, names[j]);
                strcpy(names[j], names[j + 1]);
                strcpy(names[j + 1], temp);
            }
        }
    }

    printf("\nNames in alphabetical order:\n");
    for (i = 0; i < n; i++) {
        printf("%s\n", names[i]);
    }

    return 0;
}
```

Question: Develop a C program to read N names and sort the names in reverse alphabetic order.

Answer: *(Note: This logic is identical to alphabetical sorting, except the sorting condition flips (`< 0`) to arrange strings from Z to A.)*

```
C
#include <stdio.h>
#include <string.h>

int main() {
    char names[50][50], temp[50];
    int n, i, j;

    printf("Enter the number of names: ");
    scanf("%d", &n);

    printf("Enter %d names:\n", n);
    for (i = 0; i < n; i++) {
        scanf("%s", names[i]);
    }

    // Bubble sort logic for reverse alphabetical order
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (strcmp(names[j], names[j + 1]) < 0) {
                // Swap the strings
                strcpy(temp, names[j]);
```

```
                strcpy(names[j], names[j + 1]);
                strcpy(names[j + 1], temp);
            }
        }
    }

    printf("\nNames in reverse alphabetical order:\n");
    for (i = 0; i < n; i++) {
        printf("%s\n", names[i]);
    }

    return 0;
}
```

Question: Define string.

Answer: A string constant or literal is defined as a sequence of characters enclosed in double quotes that is treated as a single data element. In C, there is no distinct primitive data type for strings; instead, strings are treated as arrays of type char that are sequentially followed and terminated by a null character '\0' to signal the end of the string.

```
Syntax:
C
char string_name[size];
```

Question: Develop a C program to check whether an entered string is a palindrome or not.

Answer: (Note: The code logic on Page 91 successfully reverses a string and tracks it using strcmp(), but it misses clearing string elements safely. Below is the clean execution of that explicit program.)

```
C
#include <stdio.h>
#include <string.h>

int main() {
    char str1[50], str2[50];
    int i, j, len, x;

    printf("Enter String1\n");
    scanf("%s", str1); [cite: 1521, 1522]

    j = 0; [cite: 1523]
    len = strlen(str1); [cite: 1524]
```

```
    // Copying elements in reverse order
    for (i = len - 1; i >= 0; i--) { [cite: 1525]
        str2[j] = str1[i]; [cite: 1527]
        j++; [cite: 1527]
    }
    str2[j] = '\0'; [cite: 1529]

    printf("The original String = %s\n", str1); [cite: 1530]
    printf("The reversed String = %s\n", str2); [cite: 1532]

    x = strcmp(str1, str2); [cite: 1534]

    if (x == 0) { [cite: 1535]
        printf("%s is a palindrome\n", str1); [cite: 1536]
    } else {
        printf("%s is not a palindrome\n", str1); [cite: 1538]
    }

    return 0; [cite: 1538]
}
```

Low Priority: Binary Search & Menu-Driven Operations

Question: Develop a C program which reads 'n' sorted elements in an array and searches for the key element using the binary search technique.

Answer: (Note: The implementation provided on Page 45 is missing closing brackets for its loop controls and code statements. The fully corrected execution tree is below.)

```
C
#include <stdio.h>

int main() {
    int a[100], n, i, low, high, mid, key, flag = 0; [cite: 713]

    printf("Enter the size of the array\n"); [cite: 715]
    scanf("%d", &n); [cite: 717]

    printf("Enter %d elements in ascending order\n", n); [cite: 719]
    for (i = 0; i < n; i++) { [cite: 720]
        scanf("%d", &a[i]); [cite: 724]
    }

    printf("Enter the element to be searched\n"); [cite: 728]
    scanf("%d", &key); [cite: 730]
```

```
    low = 0; [cite: 731]
    high = n - 1; [cite: 732]

    // Binary search logic
    while (low <= high) { [cite: 733]
        mid = (low + high) / 2; [cite: 735]

        if (key == a[mid]) { [cite: 708]
            flag = 1; [cite: 712]
            break; [cite: 714]
        }
        else if (key > a[mid]) { [cite: 719]
            low = mid + 1; [cite: 721]
        }
        else {
            high = mid - 1; [cite: 725]
        }
    }

    if (flag == 1) { [cite: 729]
        printf("Element found at position %d\n", mid + 1); [cite: 736]
    } else {
        printf("Element not found\n"); [cite: 738]
    }

    return 0; [cite: 739]
}
```

Question: Develop a menu-based C program using switch to perform any 4 string manipulation functions.

Answer: (Note: The implementation on Page 95 is structurally broken with lines bleeding out of sequence elements. Below is the cleanly unified menu program.)

```
C
#include <stdio.h>
#include <string.h>

int main() {
    int len1, len2, choice, k; [cite: 1578, 1579]
    char str1[50], str2[50]; [cite: 1580, 1581]

    printf("String manipulation functions\n"); [cite: 1582]
    printf("1. Length of a string\n"); [cite: 1583, 1584]
    printf("2. Comparing 2 Strings\n"); [cite: 1585, 1586]
```

```
    printf("3. Copying string\n"); [cite: 1587]
    printf("4. Concatenating strings\n"); [cite: 1588]

    printf("Enter 2 strings:\n"); [cite: 1589]
    scanf("%s%s", str1, str2); [cite: 1590, 1591]

    printf("Enter the choice\n"); [cite: 1592, 1593]
    scanf("%d", &choice); [cite: 1594]

    switch(choice) { [cite: 1595, 1596]
        case 1:
            len1 = strlen(str1); [cite: 1599]
            len2 = strlen(str2); [cite: 1601]
            printf("Length of string1 and string2 are %d and %d\n",
len1, len2); [cite: 1602]
            break;

        case 2:
            k = strcmp(str1, str2); [cite: 1603]
            if (k == 0) { [cite: 1604]
                printf("Strings are same\n"); [cite: 1605]
            } else {
                printf("Strings are different\n"); [cite: 1607]
            }
            break;

        case 3:
            strcpy(str2, str1); [cite: 1609]
            printf("After copying string2 contains %s\n", str2); [cite:
1610]
            break;

        case 4:
            strcat(str1, str2); [cite: 1613]
            printf("The concatenated string is %s\n", str1); [cite:
1614]
            break;

        default:
            printf("Invalid choice\n"); [cite: 1616]
            break;
    }
    return 0;
}
```

Question: Develop a C program to find the length of the string using a string manipulation function.

Answer:

```c
#include <stdio.h>
#include <string.h> [cite: 1241]

int main() {
    char str1[50]; [cite: 1244]
    int len; [cite: 1245]

    printf("Enter a string\n"); [cite: 1246]
    scanf("%s", str1); [cite: 1247]

    // Using built-in string function to evaluate size
    len = strlen(str1); [cite: 1248]

    printf("Length of the String=%d\n", len); [cite: 1249]

    return 0;
}
```

MODULE 4

Question: Define a pointer and a double pointer, and explain how they are declared and initialized.

Answer:

- **Pointer:** A pointer is a variable that holds the memory address of another variable of a similar data type. It allows direct memory access and manipulation.
    - **Declaration & Initialization:** It is declared using the indirection operator (*). It is initialized by assigning it the address of a variable using the address-of operator (&).

```c
int var = 20;
int *ip;      // Declaration [cite: 1751]
ip = &var;    // Initialization (stores address of var in ip) [cite: 1753]
```

- **Double Pointer:** A double pointer (or pointer-to-pointer) is a variable that stores the memory address of another pointer. The first pointer stores the address of the actual variable, while the second pointer stores the address of the first pointer.
    - **Declaration & Initialization:** It is declared using two asterisks ().

```
C
int var = 777;
int *ptr2 = &var;  // Single pointer [cite: 2017, 2019]
int **ptr1;        // Double pointer declaration [cite: 2018]
ptr1 = &ptr2;      // Initialization (stores address of ptr2) [cite:
2020]
```

Pointer Arithmetic Program

Question: Develop a C program to calculate the addition or subtraction of two integers using pointers.

Answer: (Note: The code on Page 26 of your PDF has scattered structural elements. Below is the cleanly unified version that accurately implements addition using pointers).

```
C
#include <stdio.h>

int main() {
    int first, second, *p, *q, sum;

    printf("Enter two integers to add\n");
    scanf("%d%d", &first, &second); [cite: 1926]

    // Assign addresses to pointers [cite: 1927, 1928]
    p = &first;
    q = &second;

    // Accessing values using the indirection operator to perform
addition [cite: 1929]
    sum = *p + *q;

    printf("Sum of the numbers = %d\n", sum); [cite: 1930]
    return 0;
}
```

Parameter Passing Mechanisms

Question: Define call by value and call by reference.

Answer:

- **Call by Value:** In this mechanism, the actual values of the arguments are copied into the formal parameters of the called function. Since these parameters are stored in separate memory locations, any modifications made inside the function remain local and do not affect the original variables in the calling function.
- **Call by Reference:** In this mechanism, the memory addresses of the variables are passed to the function instead of their values. The function uses pointers to access and alter the data directly at those addresses, meaning any updates made inside the function permanently modify the original variables.

Swapping Program

Question: Develop a C program to swap two integer numbers using call by value and/or call by reference.

Answer: (Note: Page 72 provides a call by reference swap program, but it contains formatting issues such as a missing comma in n.i.max and broken block segments. Below is the fully corrected program using **Call by Reference** which permanently swaps the variables).

```
C
#include <stdio.h>

// Function prototype declaration accepting pointer parameters [cite:
2403]
void swap(int *a, int *b);

int main() {
    int a, b;

    printf("Enter two numbers\n");
    scanf("%d%d", &a, &b); [cite: 2409]

    printf("Before Swapping\na = %d\tb = %d\n", a, b); [cite: 2410]

    // Passing the memory addresses of variables a and b [cite: 2410]
    swap(&a, &b);

    printf("After Swapping\na = %d\tb = %d\n", a, b); [cite: 2411]
    return 0;
}

// Function definition that swaps values at the provided memory
addresses [cite: 2412]
void swap(int *a, int *b) {
    int temp;
    temp = *a;   // Save value at address a [cite: 2415]
    *a = *b;     // Copy value at address b into address a [cite: 2416]
```

```
    *b = temp;    // Copy saved value into address b [cite: 2417]
}
```

Function Types

Question: Discuss any two types of functions based on arguments and return values.

Answer: The document lists four categories on Page 74. Two of those types are detailed below:

1. **Function with argument and with return value:**
    - **Concept:** The calling function passes arguments to the called function. The called function processes these values and passes a resulting value back to the calling function using the return statement.
    - **Example Syntax:** int add(int a, int b);
2. **Function with argument and without return value:**
    - **Concept:** The calling function transfers data values as arguments to the called function. The called function acts on those variables (e.g., calculates and prints a message) but has a return type of void and passes nothing back.
    - **Example Syntax:** void add(int a, int b);

Advantages of Functions

Question: Discuss the advantages of functions in C.

Answer:

According to Pages 51 and 52, the primary advantages of utilizing functions include:

- **Improved Modularity:** A massive, complex program can be split up into multiple smaller, independent blocks or modules. This makes the logic far easier to read, comprehend, and troubleshoot.
- **Code Reusability:** Instead of repeatedly retyping identical blocks of lines for varied execution branches, code can be written once inside a function and invoked multiple times by passing different arguments.
- **Reduced Workload:** Development duties can be split among engineers by breaking down a large program into separate modular functions.
- **Speed & Maintenance:** Bundling core algorithms into single locations removes structural duplication, allowing logic changes to be performed rapidly.

Dynamic Memory Calculation

Question: Develop a C program using dynamic memory to compute the sum, mean, and standard deviation of all elements stored in an array of n real numbers.

Answer: *(Note: The script provided on Page 35 has nested block bugs, an incomplete second loop definition for(i=0 i<n;i++), and completely omits utilizing the dynamic allocation*

tools explained later in your document. Below is the corrected version utilizing dynamic memory allocation (malloc/calloc) to compute the data values accurately).

C

```c
#include <stdio.h>
#include <stdlib.h> // Required for calloc/malloc and free [cite: 2099]
#include <math.h>   // Required for pow() and sqrt() [cite: 2034]

int main() {
    float *a, sum = 0, sumvar = 0, mean, var, sd; [cite: 2037]
    float *ptr; [cite: 2038]
    int n, i; [cite: 2039]

    printf("Enter the number of elements\n");
    scanf("%d", &n); [cite: 2040]

    // Allocate memory dynamically for 'n' real elements [cite: 2120]
    a = (float *)calloc(n, sizeof(float)); [cite: 2123, 2137]
    if (a == NULL) {
        printf("Memory allocation failed.\n"); [cite: 2139]
        return 1; [cite: 2140]
    }

    printf("Enter %d array elements\n", n); [cite: 2041]
    for (i = 0; i < n; i++) {
        scanf("%f", &a[i]); [cite: 2043]
    }

    // Step 1: Calculate the Sum [cite: 2028]
    ptr = a; [cite: 2045]
    for (i = 0; i < n; i++) {
        sum = sum + *ptr; [cite: 2048]
        ptr++; [cite: 2049]
    }

    // Step 2: Calculate the Mean [cite: 2028]
    mean = sum / n; [cite: 2050]

    // Step 3 & 4: Calculate Variance and Sum of Squared Distances
[cite: 2029, 2030]
    ptr = a; [cite: 2052]
    for (i = 0; i < n; i++) {
        sumvar = sumvar + (pow((*ptr - mean), 2)); [cite: 2056]
        ptr++; [cite: 2057]
    }
```

```
    var = sumvar / n;    // Variance [cite: 2058, 2031]
    sd = sqrt(var);      // Population Standard Deviation [cite: 2059]

    printf("Sum = %f\n", sum); [cite: 2060]
    printf("Mean = %f\n", mean); [cite: 2061]
    printf("Standard Deviation = %f\n", sd); [cite: 2062]

    // Free dynamically allocated space to avoid leaks [cite: 2114,
2202, 2206]
    free(a); [cite: 2208]

    return 0;
}
```

Medium Priority: Arrays, Searching, Sorting & Functions

1D Arrays & Functions: Develop a C program to find the largest or smallest element in an array by passing the array to a function.

Answer: (Note: Page 90 of the Module 4 document provides a broken skeleton for finding the largest element. Below is the fully corrected program that successfully passes an array to a function to compute the largest item).

```
C
#include <stdio.h>

// Function prototype declaration [cite: 2597]
int largest(int a[20], int n);

int main() {
    int a[20], n, i, max;

    printf("Enter the value of n\n"); [cite: 2601]
    scanf("%d", &n); [cite: 2602]

    printf("Enter %d values\n", n); [cite: 2603]
    for (i = 0; i < n; i++) { [cite: 2604]
        scanf("%d", &a[i]); [cite: 2605]
    }

    // Passing the entire array and its size to the function [cite:
2593, 2606]
    max = largest(a, n); [cite: 2606]
```

```
    printf("Largest element in array = %d\n", max); [cite: 2608]
    return 0;
}

// Function definition [cite: 2609]
int largest(int a[20], int n) { [cite: 2609]
    int max = a[0], i; [cite: 2611, 2612]

    for (i = 1; i < n; i++) { [cite: 2613]
        if (a[i] > max) { [cite: 2615]
            max = a[i]; [cite: 2616]
        }
    }
    return max; [cite: 2618]
}
```

Searching/Sorting & Functions: Develop a C program to perform binary search, selection sort, or bubble sort using functions.

Answer: (Note: Module 3 introduced the basic logic for these blocks, and Module 4 emphasizes wrapping them in functions. Below is a complete C program using a modular function to execute a **Bubble Sort**).

```
C
#include <stdio.h>

// Function prototype [cite: 513]
void bubble_sort(int a[], int n);

int main() {
    int a[50], n, i;

    printf("Enter the number of elements\n"); [cite: 551, 553]
    scanf("%d", &n); [cite: 553]

    printf("Enter %d elements\n", n); [cite: 555]
    for (i = 0; i < n; i++) { [cite: 557]
        scanf("%d", &a[i]); [cite: 561]
    }

    // Call sorting function [cite: 513]
    bubble_sort(a, n);

    printf("The sorted elements are\n"); [cite: 556]
    for (i = 0; i < n; i++) { [cite: 558]
```

```
        printf("%d\t", a[i]); [cite: 563]
    }
    printf("\n");
    return 0;
}

// Function definition for bubble sort [cite: 513]
void bubble_sort(int a[], int n) { [cite: 513]
    int i, j, temp; [cite: 513]
    for (i = 0; i < n - 1; i++) { [cite: 514]
        for (j = 0; j < n - i - 1; j++) { [cite: 515]
            if (a[j] > a[j + 1]) { [cite: 518]
                // Swap elements [cite: 518]
                temp = a[j]; [cite: 519]
                a[j] = a[j + 1]; [cite: 519]
                a[j + 1] = temp; [cite: 520]
            }
        }
    }
}
```

2D Arrays & Functions: Develop a C program to find the product of two matrices, or find the largest element in a matrix, using functions.

Answer: *(Note: Pages 60–63 in Module 3 detail raw matrix multiplication, but lack modular functions. Below is a cleanly engineered application that passes a 2D matrix to a function to compute the **largest element**).*

```
C
#include <stdio.h>

// Prototype declaration for a function accepting a 2D array matrix row
parameter
int find_matrix_largest(int matrix[10][10], int rows, int cols);

int main() {
    int abc[10][10], r, c, i, j, max;

    printf("Enter rows and columns for the matrix:\n");
    scanf("%d %d", &r, &c);

    for (i = 0; i < r; i++) {
        for (j = 0; j < c; j++) {
            printf("Enter value for abc[%d][%d]: ", i, j); [cite: 901]
            scanf("%d", &abc[i][j]); [cite: 901]
        }
```

```
    }

    // Passing the 2D array to the processing block
    max = find_matrix_largest(abc, r, c);

    printf("\nLargest element in the matrix = %d\n", max);
    return 0;
}

int find_matrix_largest(int matrix[10][10], int rows, int cols) {
    int max = matrix[0][0];
    int i, j;

    for (i = 0; i < rows; i++) {
        for (j = 0; j < cols; j++) {
            if (matrix[i][j] > max) {
                max = matrix[i][j];
            }
        }
    }
    return max;
}
```

Low Priority: Test Paper Single-Swin Syllabus

Dynamic Memory Functions: Explain the malloc(), calloc(), realloc(), and free() functions using suitable code snippets.

Answer: Dynamic memory allocation allows a program to claim system memory locations explicitly at runtime. These functions belong to the <stdlib.h> header library.

- **malloc()**: Allocates a single, massive block of memory of requested byte-size. The contents of the block are uninitialized and contain garbage values.
- C

int *ptr = (int *)malloc(5 * sizeof(int)); // Allocates 20 bytes [cite: 2095, 2096]

-
-
- **calloc()**: Allocates multiple sequential memory blocks for array elements. It automatically initializes all allocated bytes to zero.
- C

int *ptr = (int *)calloc(5, sizeof(int)); // Allocates 5 blocks of 4 bytes each [cite: 2123, 2125]

-
-
- **realloc()**: Modifies or alters the capacity of an existing, dynamically claimed storage pointer chunk without wiping active contents.
- C

ptr = (int *)realloc(ptr, 10 * sizeof(int)); // Resizes block from 20 to 40 bytes [cite: 2158, 2161]

-
-
- **free()**: Deallocates and releases previously requested heaps back to the operating system. Failing to apply this creates memory leaks.
- C

free(ptr); // Releases the block [cite: 2115, 2202, 2208]

-
-

Function Anatomy: Explain the different parts of a user-defined function.

Answer: A user-defined function is broken down into three crucial architectural parts:

1. **Function Declaration / Prototype**: Informs the C compiler ahead of time about the name of the function, its parameter types, and its expected return category.
    - *Syntax:* return_type function_name(argument_list);
2. **Function Call**: Invokes the logic execution step from a main tracking routine using matching actual parameters.
    - *Syntax:* function_name(argument_list);
3. **Function Definition**: Contains the operational body with local variable definitions and statement logic executing the dedicated programming workload.

Syntax:

```
return_type function_name(argument_list) { [cite: 2299, 2300]
    local_variable_declaration; [cite: 2301]
    // Body of function [cite: 2302]
} [cite: 2301]
```

    -

Basic Function Application: Develop a C program to calculate the perimeter of a rectangle using functions.

Answer: *(Note: Page 88 features a functional tracking snippet for this calculation. It has been refined below to ensure a clean compilation layout).*

```
C
#include <stdio.h>

// Function prototype [cite: 2579]
int perimeter(int x, int y);

int main() {
    int l, b, p; [cite: 2582]
```

```
    printf("Enter length and breadth\n"); [cite: 2584]
    scanf("%d%d", &l, &b); [cite: 2585]

    // Function call using actual parameters 'l' and 'b' [cite: 2586]
    p = perimeter(l, b);

    printf("Perimeter of Rectangle = %d\n", p); [cite: 2586]
    return 0;
}

// Function definition with formal parameter markers [cite: 2587]
int perimeter(int x, int y) { [cite: 2587]
    int per; // Local variable [cite: 2589]
    per = 2 * (x + y); [cite: 2590]
    return per; // Passes value back to caller loop [cite: 2340, 2591]
}
```